

□ Doubly Linked List Operations

□ Aim

To implement the following operations on a **Doubly Linked List**:

1. Insert at front
2. Insert at end
3. Delete last node
4. Delete node before a specified position

CODE:

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node *prev, *next;
};

struct Node* head = NULL;

// Insert at Front
void insertFront(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL)
        head->prev = newNode;

    head = newNode;
}

// Insert at End
void insertEnd(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    struct Node* temp = head;

    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL) {
        newNode->prev = NULL;
```

```

    head = newNode;
    return;
}

while (temp->next != NULL)
    temp = temp->next;

temp->next = newNode;
newNode->prev = temp;
}

// Delete Last Node
void deleteLast() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node* temp = head;

    if (temp->next == NULL) {
        free(temp);
        head = NULL;
        return;
    }

    while (temp->next != NULL)
        temp = temp->next;

    temp->prev->next = NULL;
    free(temp);
}

// Delete node before given position
void deleteBeforePosition(int pos) {
    if (head == NULL || pos <= 1) {
        printf("Invalid position\n");
        return;
    }

    struct Node* temp = head;
    int i = 1;

    while (i < pos && temp != NULL) {
        temp = temp->next;
        i++;
    }

    if (temp == NULL || temp->prev == NULL) {

```

```

    printf("Deletion not possible\n");
    return;
}

struct Node* del = temp->prev;

if (del->prev != NULL)
    del->prev->next = temp;
else
    head = temp;

temp->prev = del->prev;

free(del);
}

// Display List
void display() {
    struct Node* temp = head;
    if (temp == NULL) {
        printf("List is empty\n");
        return;
    }

    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main Function (Menu)
int main() {
    int choice, data, pos;

    while (1) {
        printf("\n--- Doubly Linked List Menu ---\n");
        printf("1. Insert at Front\n");
        printf("2. Insert at End\n");
        printf("3. Delete Last Node\n");
        printf("4. Delete Node Before Position\n");
        printf("5. Display\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value: ");

```

```

        scanf("%d", &data);
        insertFront(data);
        break;

    case 2:
        printf("Enter value: ");
        scanf("%d", &data);
        insertEnd(data);
        break;

    case 3:
        deleteLast();
        break;

    case 4:
        printf("Enter position: ");
        scanf("%d", &pos);
        deleteBeforePosition(pos);
        break;

    case 5:
        display();
        break;

    case 6:
        exit(0);

    default:
        printf("Invalid choice\n");
    }
}

return 0;
}

```

□ Sample Run

```

--- Doubly Linked List Menu ---
1. Insert at Front
2. Insert at End
3. Delete Last Node
4. Delete Node Before Position
5. Display
6. Exit
Enter your choice: 1
Enter value: 10
Enter your choice: 2
Enter value: 20
Enter your choice: 5
10 <-> 20 <-> NULL

```

□ Conclusion

- Doubly linked list allows **bidirectional traversal**
- Insertions and deletions are efficient
- Requires extra memory for `prev` pointer